

Executive summary

Why

The AI Failure Museum is an online app that teaches users about the times AI systems have failed to make good decisions about the environment. As AI is used more and more for things like predicting flood danger, managing land use, and keeping an eye on biodiversity, there is a genuine risk that people may trust these systems too much without knowing what they can and can't do. Our initiative solves this problem by showing case studies of AI failures in a clear, systematic way. The goal is to enable people, especially students and professionals, to have a more critical and informed view of AI so they don't just take what it says without thinking about it in real life.

Good things

The best thing about the platform is that it helps people learn from their mistakes instead of making them again. Some of the benefits are:

- Users can look through and read case studies that are like real life and explain what went wrong, why it happened, and what could have been done better.
- Reflective tests have people think about the problems instead of merely reading about them, which helps them remember what they've learned.
- Curators, like researchers or teachers, can add and manage displays, which means the museum can grow over time.
- The gamification system (points and badges) gives users a tiny reason to get more involved with the content.

Main Risks

We have considered about a few major risks:

Spreading false information or bias

There is a chance that people may think the museum is against AI because our case studies talk about AI failures. There is also a chance that the way we talk about failures will show our own prejudices. We have tried to deal with this by authoring case studies that focus on what went wrong instead of who is to blame, and by making it clear that all information is meant to be examples, not true stories about real businesses. But we need to keep this in mind as additional content is added.

Protecting and securing data

The system gathers basic personal information from users who sign up, such as their email addresses and passwords. This information may be made public if there was a security breach. We have done things to lower this risk. For example, passwords are hashed and never saved in plain text, secret keys are kept out of GitHub using a .env file, and role-based access control keeps people from getting to areas of the system they shouldn't. That being said, security is still a worry, and we expect to undertake more extensive testing in Sprint 2.

Dataset doesn't fulfil the basic standards

The criteria says that there must be at least 20 exhibits, 40 quiz questions, and 5 ways to fail. We are working toward these goals, but we haven't entirely attained them for CW1 yet. Luqman, our scrum master, is keeping track of progress, and this is the biggest risk to delivery as we head into the last push.

What the Prototype Currently Achieves

At this point, the prototype gives us a workable vertical slice of the system. In particular:

- Users can look through a collection of AI failure exhibits, each of which has an explanation of what went wrong, what led to it, and what they learned from it.
- There is a simple quiz linked to the displays that lets people see how well they comprehend the failure.
- Users get points and badges for finishing quizzes.
- Users can sign up and log in, and visitors and curators have different duties.
- Curators can use a dashboard to make, change, and delete exhibits.
- There is a privacy policy and a way to remove my data.
- You can run the system locally with a seeded dataset, so it works for marking even if the online site has problems.

Project description and prioritised requirements (around 1000 words)

Problem statement:

Systems that depend on AI are starting to be increasingly utilized in important situations, from high stakes environmental domains (deforestation, biodiversity monitoring) to evaluating risk to human life (flood maps). Failures sometimes occur in these systems, often the same mistakes repeated - this points to the lack of resources that document such failures for people to learn from. This AI-Museum curates publicly accessible information regarding AI failures in such domains to make lessons and learning more accessible for the average person, policy makers, governments, organisations, researchers, or anyone whom it may concern.

What is in VS out of the scope:

The project's goal is to document the environmental-impacting AI failures, encouraging users to interact with quiz-based learning and a points system. It also aims to provide tools to help curators curate the content on the platform. However it is not intended to provide real-time AI monitoring, nor legal liability tracking or non environmental AI failures.

1.1 Core Requirements

3.1.1: System displays a browsable collection of AI failure exhibits

Users can freely explore a list or gallery of documented AI failure cases.

3.1.1: System provides search functionality across exhibits

Users can search exhibits using keywords (e.g. system name, domain, failure type).

3.1.2: Each exhibit page displays required sections

Every exhibit clearly presents context, purpose, claimed outputs, failure details, causes, and lessons learned.

3.1.3: System displays timeline/sequence of failure emergence

Each exhibit shows a chronological or step-by-step view of how the failure developed over time.

3.1.4: System allows access to supporting artefacts

Exhibits include related materials such as screenshots, diagrams, logs, or summaries.

3.1.5: Interface is mobile friendly and accessible

The UI adapts to different screen sizes and meets basic accessibility standards.

3.1.1: System supports filtering by domain

Users can narrow exhibits by application domain (e.g. healthcare, finance, education).

3.1.1: System supports filtering by failure category

Users can filter exhibits by types of AI failure (e.g. bias, robustness, misuse).

1.2 Data Model Requirements

3.2: Exhibits include core metadata

Each exhibit stores title, domain, deployment context, and intended use.

3.2: AI system descriptions include technical details

Each exhibit explains system type, inputs/assumptions, and presented outputs.

3.2: Failure descriptions include impact details

Each failure records what went wrong, how it was detected, and who was affected.

3.2: Contributing factors are categorised

Causes are grouped into clear categories such as data, design, or governance issues.

3.2: Lessons learned include recommendations and warnings

Each exhibit provides actionable advice and cautionary guidance.

4.1: Supports at least 5 failure categories

The system defines and uses a minimum of five distinct failure types.

4.1: Contains at least 20 exhibits

The system includes a sufficiently large dataset for meaningful exploration.

4.1: Contains at least 20 supporting artefacts

There are multiple artefacts distributed across exhibits.

4.1: Contains at least 40 quizzes

The system includes a substantial number of assessment questions.

1.3 Learning and Interaction Requirements

3.3: System provides short quizzes linked to exhibits

Each exhibit includes one or more short quizzes tied to its content.

3.3: Quizzes test understanding of causes and assumptions

Questions assess reasoning about why the failure occurred, not just recall.

3.3: System provides reflection prompts

Users are prompted to think critically about uncertainty, bias, and appropriate AI use.

3.3: System includes a points system

Users earn points for engaging with exhibits, quizzes, and reflections.

3.3: System awards badges

Achievements are recognised with visible badges for milestones or skills.

3.3: UI is appropriate for a university context

Design is professional, clear, and not overly gamified or distracting.

2.1: Visitors can bookmark exhibits

Users can save exhibits to revisit later.

3.3: System provides quiz explanations after answers

Users receive feedback explaining why answers are correct or incorrect.

Curator Tool Requirements

3.4: Curators can create new exhibits

Authorized users can add new AI failure cases to the system.

3.4: Curators can edit existing exhibits

Curators can update exhibit content as needed.

3.4: Curators can archive exhibits

Exhibits can be hidden from public view without deletion.

3.4: Curators can upload supporting artefacts

Curators can attach files or media to exhibits.

3.4: Curators can manage artefacts

Artefacts can be reordered, edited, or removed.

3.4: Curators can create quizzes

Curators can author quiz questions linked to exhibits.

3.4: Curators can edit quizzes

Existing quizzes can be updated or corrected.

3.4: Curators can review visitor feedback or flags

Curators can see reported issues or comments from users.

3.4: Curators can view engagement analytics

Curators can access basic usage statistics.

3.4: Analytics show most viewed exhibits

The system identifies which exhibits receive the most traffic.

3.4: Analytics show quiz completion rates

The system tracks how often quizzes are completed.

Privacy and Data Protection Requirements

3.6: System has a clear privacy policy

Users can easily understand how their data is handled.

3.6: System minimises personal data collection

Only essential user data is collected.

3.6: System uses pseudonymous display names

User identities are not publicly exposed.

3.6: System provides a “delete my data” mechanism

Users can request deletion of their stored data.

3.6: System avoids naming and shaming real people or businesses

Content focuses on systems and failures, not individuals.

Technical and Deployment Requirements

4: System works locally with seeded data

The application runs without internet access using preloaded data.

2.3: System is deployable for offline demonstration

The system can be shown in demos without external services.

3.5: No secrets committed to Git

Sensitive credentials are excluded from version control.

3.5: Environment variables documented via `.env.example`

Required configuration variables are clearly documented.

2.3: System has documented local setup steps

Clear instructions exist for installing and running the system locally.

3.5: Repository shows visible team contributions

Version history reflects contributions from multiple team members.

2.3: Exhibit schema is extensible

The data model can support new failure types without major refactoring.

Evaluation Requirements

5: System tracks comprehension via quiz performance

Quiz results are used to measure user understanding.

5: System supports analysis of reflection quality

Reflections can be reviewed or assessed for depth and relevance.

5: System tracks engagement metrics

The system records metrics such as time spent and revisit frequency.

5: System is usable with assistive technologies

The application supports screen readers, keyboard navigation, and similar tools.

Success Criteria

Our evaluation is structured around five key questions we want to answer by the end of the project:

- **Comprehension:** Can users grasp the website's aim and why AI systems failed in every exhibit they read?
- **Reflection:** Do quizzes allow users to reflect on their learning?
- **Accessibility:** Can users with impairments such as colour blindness use all core features?
- **Usability:** Can curators effectively create and manage exhibits without developer help?
- **Engagement:** Points and badges: do they boost engagement without trivialising context?

The table below sets out the specific criteria, metrics, targets, and how we plan to measure each one:

Criterion	Metric	Target	Measurement method
Comprehension	Quiz accuracy (average score across all attempts)	> 60% average	Analytics tracking total correct answers vs total answers submitted
Reflection	Users shown correct answer feedback after every quiz submission	100% of quiz attempts	Manual check that feedback displays correctly after every quiz submission
Engagement	Quiz completion rate (quiz attempts ÷ exhibit page views)	> 60%	Analytics tracking exhibit page views and quiz attempt counts
Usability	Curator can create a new exhibit without developer help	Task completed successfully	Task completion test — ask a curator to add an exhibit from scratch and observe
Accessibility	WCAG 2.1 AA compliance across all core pages	Pass on all core pages	Automated audit using Axe or Lighthouse, plus manual testing with a screen reader
Content	Number of exhibits in the database	At least 20 exhibits	Count in the database or on the website
Technical	Page load time	Under 2 seconds	Lighthouse performance audit
Technical	Mobile usability score	Lighthouse score above 50	Lighthouse mobile audit

Notes on targets

- A 60% quiz accuracy rate is a good starting point because it shows that users are getting the information without it being too simple to be useful.
- A 60% completion rate is normal for engagement metrics on educational sites.
- The requirements paper says that WCAG 2.1 AA is the standard we need to follow, thus that's the proper goal to set.
- A Lighthouse mobile score of over 50 is a good goal for Sprint 1. It's not ideal, but it shows that the site works on mobile.

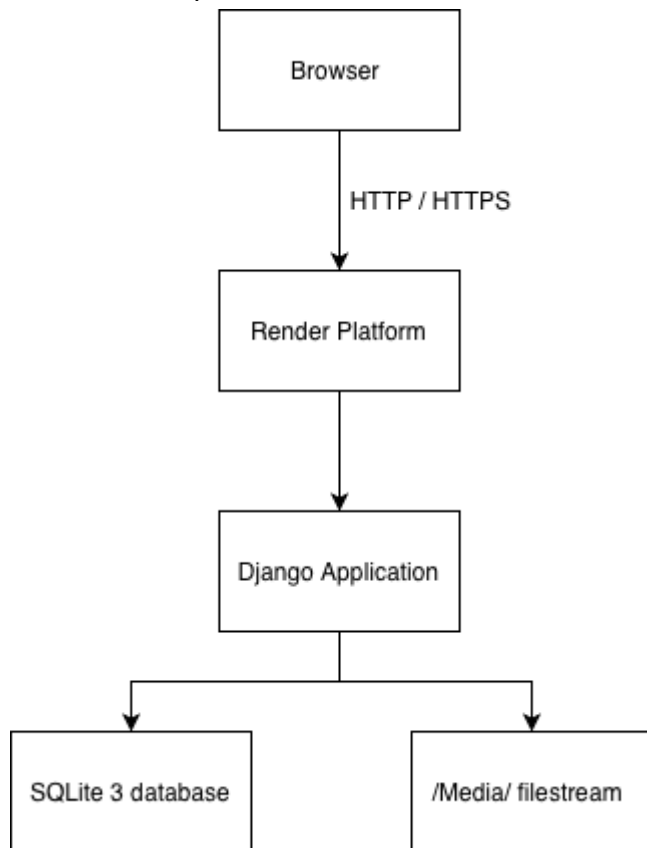
Architecture (v1)

Overview

The AI Failure museum is a django based web application deployed using Render. It follows a 'server side' architecture; in which django handles the request routing, html rendering and logic - SQLite3 provides data storage and management.

Component diagram

The diagram below shows a high level topology of the system, showing the traffic flows through Render into django. In local development Render isn't involved.



Component description

Browser (client)

The browser in this diagram represents the end user's web browser. It issues HTTP protocol GET/POST requests and receives fully rendered HTML pages in response. All page rendering is server side. The browser also fetches static assets like CSS, images etc.

Render (cloud platform)

The host that deploys the site. Render manages the HTTP(S) termination and domain routing (using [.onrender.com](https://render.com)). It also manages environment variable injection (things like DEBUG and SECRETKEY). It also process the lifecycle. The application is configured the render.yaml

Gunicorn (Green Unicorn)

The WSGI application server used to run the Django application in production on Render. It sits between Render's HTTP(S) routing layer and the Django application, receiving incoming HTTP requests and forwarding them to Django for processing. Gunicorn manages multiple worker processes, allowing the application to handle concurrent requests efficiently and improving reliability under load. It does not perform request routing or HTML rendering itself; instead, it executes the Django WSGI entry point. In local development, Django's built-in server replaces Gunicorn, but in production Gunicorn provides a stable, production-ready runtime.

Django Application

The core of the system, consisting of sub components listed below:

- URL Router ([url.py](#)): maps HTTP paths to the correct view functions. It handles public routes, curator routes and authentication routes and REST API routes.
- Views ([views.py](#)): Contains the request handling logic. It queries the database via Django's built ORM. It applies RBAC rules before curator actions, does point calculation etc and returns the rendered HTML (or JSON) responses
- Templates ([templates.py](#)): Server side HTML files populated by Django's template engine. We organised it into sub folders to store curator interfaces, quiz interfaces, user registration and authentication etc.
- Gamification Module ([gamification.py](#)): Awards points (and theoretically badges) to the user after a quiz submission
- RBAC Module ([rbac.py](#)): Role-based access control helper lightweight module, categorizes whether the authenticated user belongs in the 'curator' group before permitting access.

SQLite3 Database

A relational database managed through Django's ORM. It stores all application data like exhibits, categories, quizzes, questions and answer options, user accounts, quiz attempts, responses, points, badges. It is located in `4_code/djangoCode/db.sqlite3`. It is pre seeded with basic demo data for local hosting, called via python [manage.py](#) `loaddata`

Media File Storage (/media/)

A directory that is local that stores files uploaded by curators when creating exhibits (any uploaded artefact). Django streams these on requests. This is currently local disk storage, in V2 we will produce multi-instance deployment.

=====

Data model and dataset description:

Category (one) -> (many) Exhibit : each exhibit has one domain category

Exhibit (one) -> (many) Quiz : each exhibit can have many quizzes

Quiz (one) -> (Many) Questions : each quiz has many questions

Questions (one) -> (Many) AnswerOptions : each question has multiple answer options

User (one) -> (one) UserProfile : each user has one account

User (one) -> (many) QuizAttempt each user can do a quiz however much they want

Category		
int	id	PK
string	name	UK
string	slug	UK
text	description	

contains

Exhibit		
int	id	PK
string	title	
int	category_id	FK
string	failure_type	
text	deployment_context	
text	intended_use	
date	incident_date	

has

Quiz		
int	id	PK
int	exhibit_id	FK
string	title	
text	description	
boolean	is_active	

includes

Question		
int	id	PK
int	quiz_id	FK
text	prompt	
enum	question_type	
int	order	

offers

has_correct

AnswerOption		
int	id	PK
int	question_id	FK
text	answer_text	
boolean	is_correct	
int	order	

User		
int	id	PK
string	username	
string	email	

has

makes

UserProfile		
int	id	PK
int	user_id	FK
text	bio	
date	join_date	

QuizAttempt		
int	id	PK
int	user_id	FK
int	quiz_id	FK
datetime	attempt_date	
int	score	

Evaluation plan (initial):

The evaluation of V1 is structured around 5 main points:

1. Comprehension - do users correctly understand the AI failure after reading an exhibit
2. Reflection: do users engage meaningfully with questions about the failure
3. Engagement - do users who view an exhibit go on to complete the quiz?
4. Accessibility - does the site meet WCAG standards for all user groups?
5. Usability - can the curator create a new exhibit without developer assistance

So far we have implemented the Reflection which allows the users to see their results after they complete a quiz. They are shown if they got the right answer or if they got the wrong one. If they were incorrect they are shown the correct answer. We also have Usability implemented which allows for curators to interact with the exhibits and quizzes without any developer assistance.

We currently only have Reflection and Usability implemented. While users can take the quizzes we are unable to track comprehension as we haven't built a way to track quiz performances. Similarly we haven't yet implemented a way to track how many times an exhibit is revisited or how much time is spent per exhibit to check Engagement. For Accessibility we tried our best to follow the WCAG standards however some pages have some text contrast issues that need to be solved.

Sprint 2 plan:

For comprehension we plan on implementing a way to track overall quiz performance based on time spent on the exhibit, this will allow us to see how long it would take on average to fully understand the exhibits.

For Engagement we plan on adding a page view count for exhibits and quiz completion rates. We'll calculate the quiz completion rate by taking the quiz attempt count divided by the page view count. Our aim is for the completion rate to be above 60%.

For Accessibility we are planning on running an automated WCAG audit to see how we stand against the standards and improve upon any of the standards we fall behind on.

We currently have around 6 exhibits with 5 categories. We plan on expanding the number of exhibits to a minimum of 20 alongside increasing the number of quizzes to a minimum of 40.